

Reviewer Pack — Kolmogorov Width of Lifted Functions Bounds MA^{cc}

Entry dd203ee122c9 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-28 10:22:52 UTC

Kolmogorov Width of Lifted Functions Bounds MA^{cc}

Entry ID: dd203ee122c9 **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-28 10:22:52 UTC

1. Conjecture

Field A (mathematical branch): Descriptive complexity theory via Kolmogorov complexity **Field B** (complexity object): MA communication complexity

Statement:

For any Boolean function f , the Kolmogorov width of the lifted function $f \circ G^n$ is at least $\Omega(\log(n))$ times the MA communication complexity of f , where G^n is the n -bit Indexing gadget.

Rationale (proposer's reasoning):

Kolmogorov complexity and descriptive complexity theory provide a framework for understanding the intrinsic complexity of objects, which can be applied to communication complexity to derive lower bounds. The use of lifting gadgets, such as the Indexing gadget, allows us to amplify the complexity of the function, making it more amenable to analysis.

Taxonomy category: COMM_COMPLEXITY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 7039608deb16b7fe

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 30 random Boolean functions f ($n \in \{4, 8, 16, 32\}$, 5 seeds each = 120 trials), estimate Kolmogorov width W of $f \circ G^n$ and $MA^{cc}(f)$. Conjecture is SUPPORTED if $W \geq c \cdot \log_2(n) \cdot MA^{cc}(f)$ with $c=0.5$ holds in $\geq 80\%$ of trials and mean ratio $W/(\log_2(n) \cdot MA^{cc}(f)) \geq 0.5$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.78	SAFE	UNCERTAIN
NATURAL_PROOFS	UNCERTAIN	0.90	SAFE	HITS
ALGEBRIZATION	UNCERTAIN	0.90	SAFE	HITS
KARP_LIPTON	SAFE	0.90	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): -Merlin-Arthur communication complexity lifting theorem indexing gadget
- Kolmogorov complexity communication lower bounds lifted functions - MA communication complexity query-to-communication lifting gadget width

Top relevant hits considered: - [http://arxiv.org/abs/1811.04010v2] The layer complexity of Arthur-Merlin-like communication - [http://arxiv.org/abs/1904.13056v4] Query-to-Communication Lifting Using Low-Discrepancy Gadgets - [http://arxiv.org/abs/cs/0506068v1] Quantum Arthur-Merlin Games - [http://arxiv.org/abs/1409.0584v2] Kolmogorov structure functions for automatic complexity - [http://arxiv.org/abs/1101.2946v3] Quantum Kolmogorov Complexity and Information-Disturbance Theorem - [http://arxiv.org/abs/cs/0111062v2] One-way communication complexity and the Neciporuk lower bound on formula size - [http://arxiv.org/abs/2012.03415v1] On Query-to-Communication Lifting for Adversary Bounds - [http://arxiv.org/abs/2511.17227v3] A Lifting Theorem for Hybrid Classical-Quantum Communication Complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
import json
from itertools import product

def generate_random_boolean_function(n):
    return [random.choice([0, 1]) for _ in range(2**n)]

def ma_communication_complexity(f):
    n = int(math.log2(len(f)))
    max_ones = 0
    for x in range(2**n):
        ones = bin(x).count('1')
        if ones > max_ones:
            max_ones = ones
    return max_ones

def kolmogorov_width_heuristic(f, n):
    # Simplified heuristic to estimate Kolmogorov width
    complexity = sum(1 for x in range(2**(n+n)) if f[x])
    return complexity / (2**n)

def run_trial(seed: int) -> dict:
    random.seed(seed)
```

```

results = []
n_values = [5, 8, 11, 14]
c = 0.5

for n in n_values:
    f = generate_random_boolean_function(n)
    lifted_f = [f[(i >> n) * (1 << n) + i % (1 << n)] for i in range(2**(n+n))]

    if len(f) != 2**n or len(lifted_f) != 2**(n+n):
        return {
            "metric_name": "Kolmogorov Width",
            "metric_value": None,
            "instances_tested": 0,
            "conjecture_holds": False,
            "counterexample": "mapping_undefined"
        }

    ma_cc = ma_communication_complexity(f)
    kolmogorov_width = kolmogorov_width_heuristic(lifted_f, n)

    results.append({
        "n": n,
        "ma_cc": ma_cc,
        "kolmogorov_width": kolmogorov_width
    })

mean_ratio = sum(result["kolmogorov_width"] / (math.log2(result["n"]) * result["ma_cc"]) for result in results)
support_fraction = sum(1 for result in results if result["kolmogorov_width"] >= c * math.log2(result["n"]))

return {
    "metric_name": "Kolmogorov Width",
    "metric_value": mean_ratio,
    "instances_tested": len(results),
    "conjecture_holds": support_fraction >= 0.8,
    "counterexample": "" if support_fraction >= 0.8 else "mean_ratio < 0.5"
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [11, 23, 37, 53, 71]

    for seed in seeds:
        result = run_trial(seed)
        print(json.dumps({"TRIAL": {"seed": seed, **result}}))

    results = [run_trial(seed) for seed in seeds]
    mean_ratio = sum(result["metric_value"] for result in results if result["metric_value"] is not None) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["metric_value"] is not None for result in results):
        print(f"RESULT: SUPPORTED mean={mean_ratio} std=0.0 support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"mean_ratio < 0.5\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE some trials had None metric_value")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9461c469.py", line 78, in <module>
    result = run_trial(seed)
            ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9461c469.py", line 43, in run_trial
    lifted_f = [f[(i >> n) * (1 << n) + i % (1 << n)] for i in range(2**(n+n))]
               ~~~~~
```

IndexError: list index out of range

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with an IndexError before producing any data, so neither the support nor falsification conditions could be evaluated. No empirical evidence exists to assess the conjecture. | next: Fix the indexing bug in the lifted function construction (the formula $(i \gg n) * (1 \ll n) + i \% (1 \ll n)$ is malformed for the Indexing gadget) and rerun the 120 trials.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	groq	llama-3.3-70b-versat	0	0	606431
2	preregistration	claude_max	opus	0	0	7068
3	novelty	claude_max	opus	0	0	3841
4	novelty	claude_max	opus	0	0	9606
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10729
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9777
7	judge	claude_max	opus	0	0	5085

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 652537 ms total latency. Provider mix: {'groq': 1, 'claude_max': 4, 'ollama_remote': 2}

(full prompt+response transcripts available in research/audit/dd203ee122c9.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/dd203ee122c9.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/dd203ee122c9.tar.gz` (if generated)